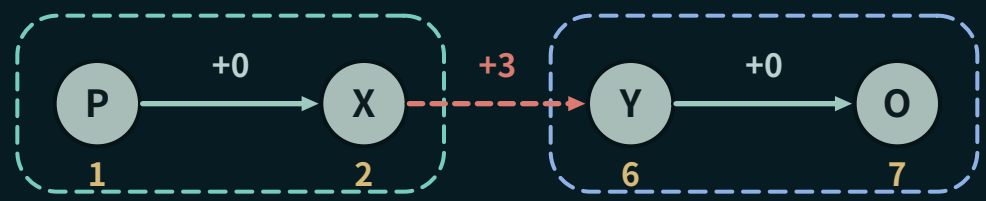
The Cost of the Critical Path

Rajaraman–Wong delay-oriented clustering · delay, replication and instances

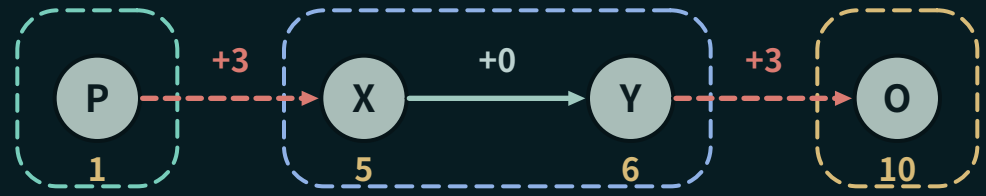
Learning Clustering · original author muranfei (Issue #8) · mapping to Lab 09 (Multilevel) in the teacher's catalogue pending

The goal has changed: from cut to delay

- Chain P→X→Y→O: δ = 1, inter-cluster D = 3, capacity 2
- {P,X} | {Y,O}: 1 crossing, delay 7
- {P} | {X,Y} | {O}: 2 crossings, delay 10
- Delay follows how often the critical path crosses a boundary



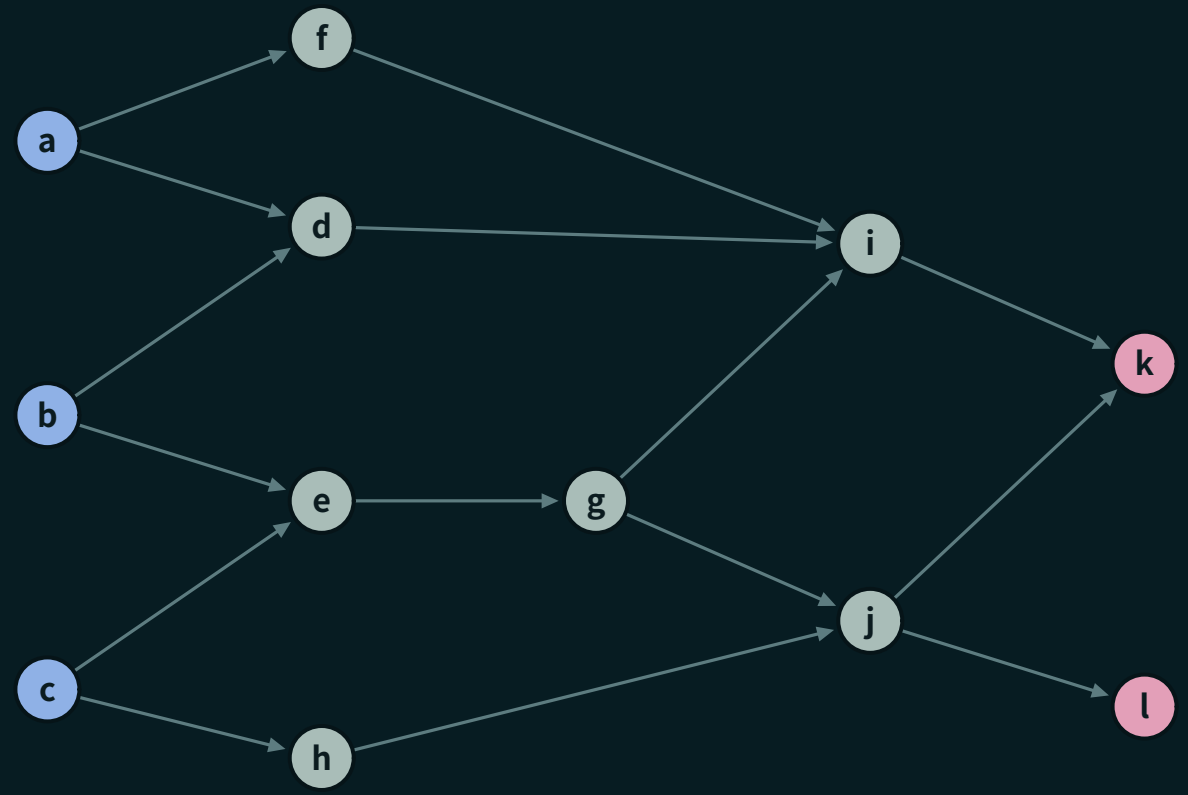
1 crossing: delay 7



2 crossings: delay 10

The RW model

- A combinational DAG; PIs are inputs, POs outputs
- Node delay δ(v); 0 on wires inside a cluster, D between
- Cluster capacity M (the sum of node sizes)
- Logic nodes may be replicated
- Goal: minimise the largest PI→PO delay

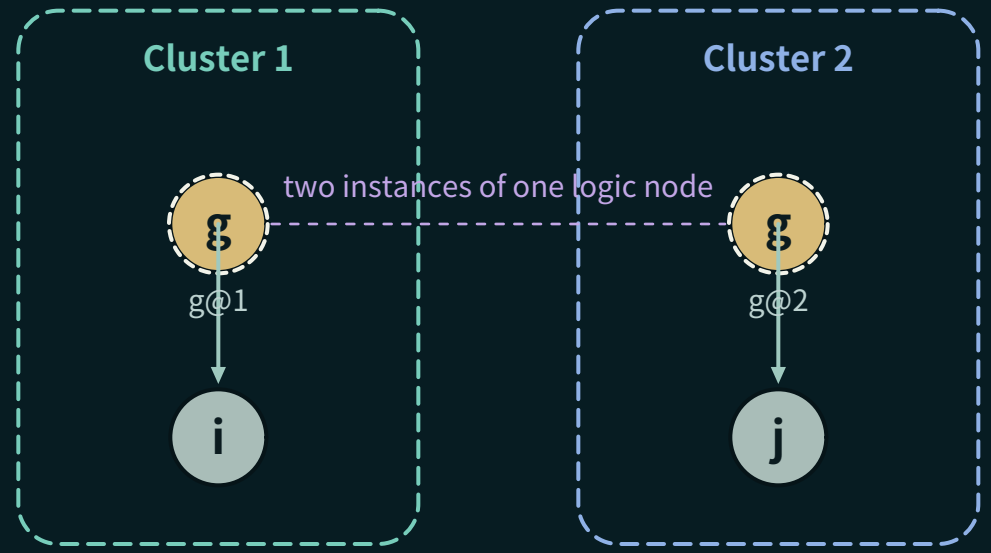


Here D is the inter-cluster delay, not Module 06's degree matrix.

Candidate data, reconstructed • tie rule assumed

Replication: logic nodes and instances

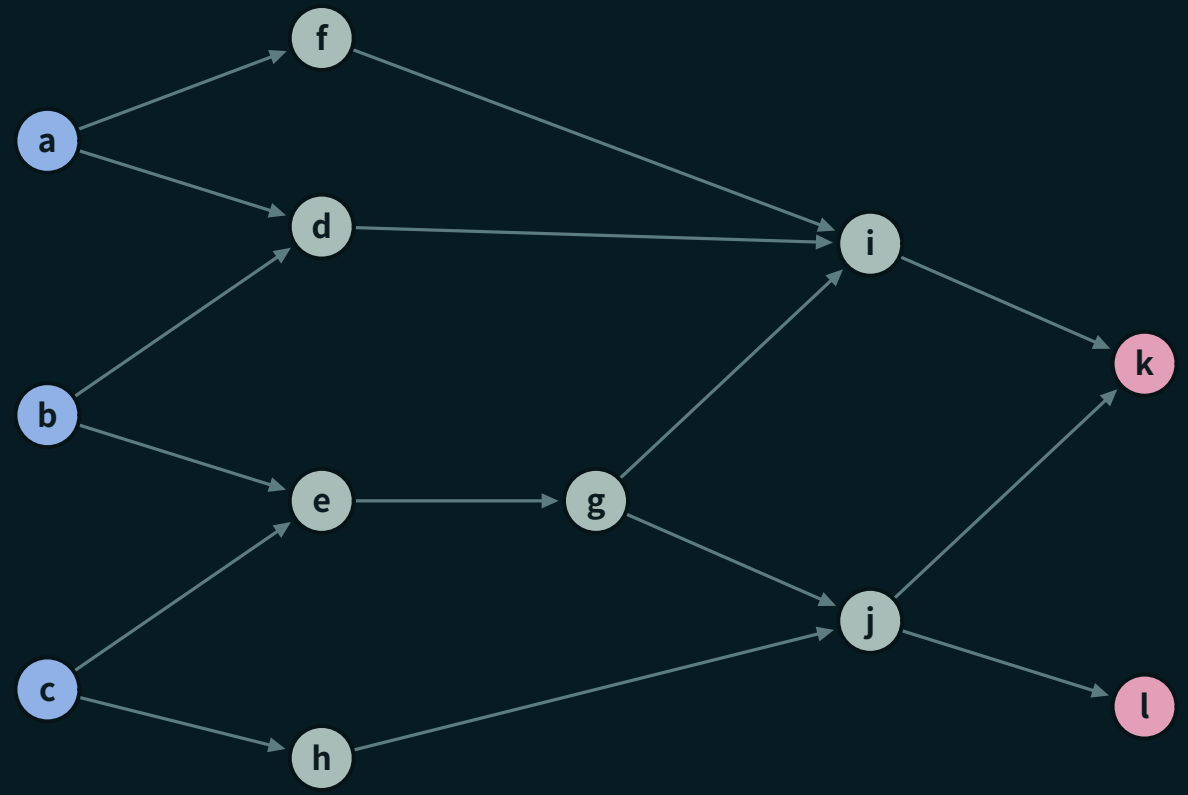
- One logic node (logicalId) may have several physical instances (instanceId)
- An instance is written "node@root"
- Replication lets several clusters get the same signal inside
- The cost: every copy takes area
- A "node → cluster" map cannot express replication



The cost: one more instance, one more unit of area (schematic)

Initialisation: the candidate DAG and Δ

- The original 12-node example: PIs a, b, c; POs k, l
- δ = 1, D = 3, capacity 4
- Δ(x, v): the longest node delay from x to v (counting v, not x)
- Δ(b, k) = 4 (b→e→g→i→k)
- The graph is reconstructed from the original film: candidate data



Candidate data, reconstructed • tie rule assumed

Labelling: l(v) = max(l1, l2)

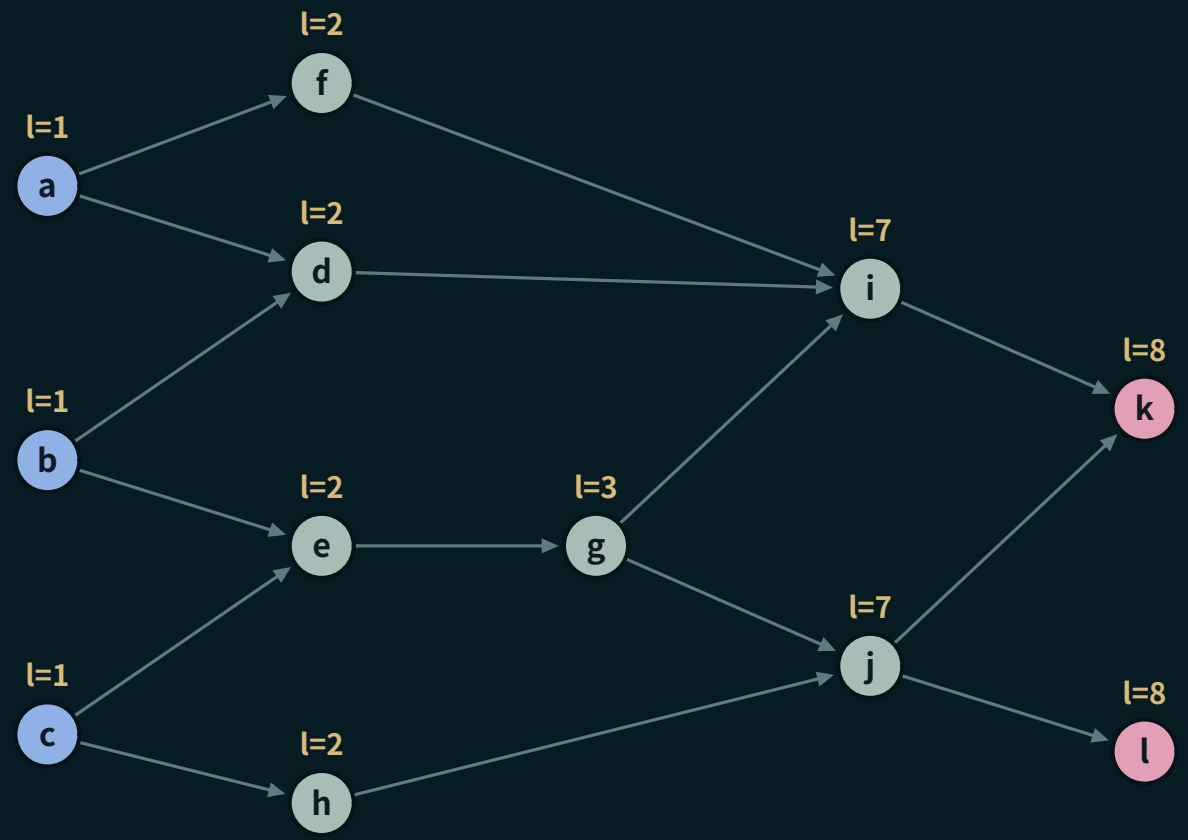
- Score(x) = l(x) + Δ(x, v)
- Candidate cluster: v + the highest-scoring ancestors, up to capacity
- Ties: the later in topological order (this course's assumption)
- l1 = largest score of a PI inside; l2 = largest score outside + D
- i: l1 = 4, l2 = 7 → l(i) = 7

Ancestor	l(x)	Δ	Score	
g	3	1	4	inside
e	2	2	4	inside
c	1	3	4	inside
b	1	3	4	outside
f	2	1	3	outside
d	2	1	3	outside
a	1	2	3	outside

l1 = 4, l2 = 7, l(i) = 7

Every label

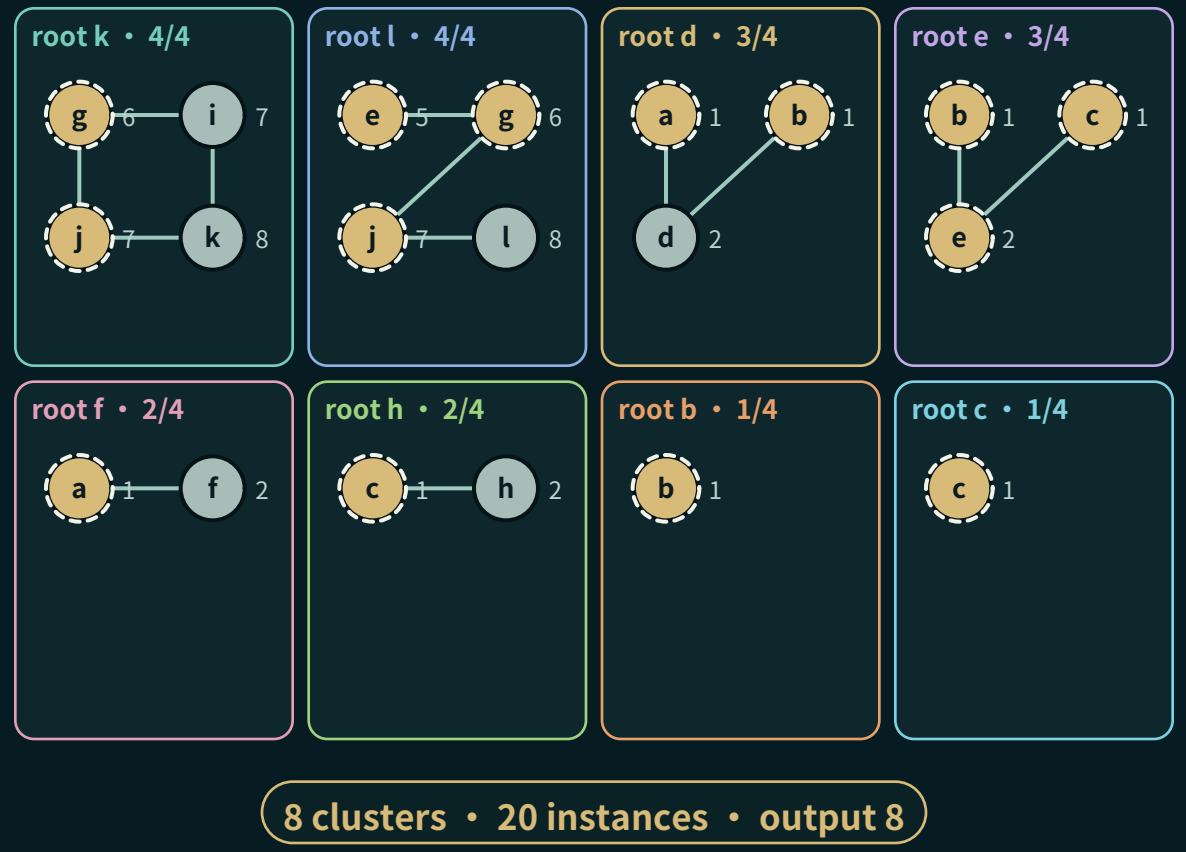
- PIs: l = 1
- d, e, f, h: 2; g: 3
- i, j: 7
- k, l: 8
- RW's labels are times, not groups



Candidate data, reconstructed • tie rule assumed

Recovery: working back from the outputs

- Take the candidate clusters of k and l; outside inputs become new roots
- 8 clusters in all: k, l, d, e, f, h, b, c
- 20 instances; g, j, e, a have 2 each, b and c 3 each
- Every instance has complete inputs
- Output arrival 8 = the labels



What replication buys

- With replication: delay 8, 20 instances
- Without: exhaustive search over every capacity-legal grouping, best 10
- 8 more instances buy 2 units of delay
- The optimality result assumes replication is allowed

	Replication (RW)	No replication (exhaustive)
Largest delay	8	10
Instances / area	20	12

One best grouping without copies: adfh | bceg | ijkl

Pseudocode: labelling and recovery

- Labelling: topological order; scores, candidate cluster, l1, l2 (Python: rw_label)
- Recovery: back from the POs; outside inputs become new roots (rw_reconstruct)
- Instances m@r; +0 inside, +D between
- Arrival times: evaluate_instance_delay

```
queue ← every PO
while queue is not empty:
  r ← pop; select r's cluster
  outside input u → queue (new root)
for each selected root r, member m:
  make instance m@r
  input inside: u@r → m@r (+0)
  input outside: u@u → m@r (+D)
```

Limits and open questions

- The optimality comes from the paper (IEEE TCAD 1995; DAC 1993) and holds only within its model
- The 12-node example is a candidate reconstruction; the tie rule is an assumption awaiting the author's confirmation
- This course's tests check consistency and legal instances; they do not prove optimality
- Submitted as Lesson 9, while the teacher's Lab 09 is Multilevel: mapping awaits the teacher
- Replication costs area; pins, power and more are outside this model

Summary

The goal becomes delay: each crossing adds D to the path.

Labelling gives times, recovery works back from the outputs; logic nodes apart from instances, and replication costs area.

Back to the comparison on the home page: eight modules, eight different operations.